有限内存下的关系图训练

RelConnector 系统优化演进 | vanilla → v1 → v2

从减少重复工作
到提高特征供给能力

01 按需读取

02 分层复用

03 有界并行

中文技术汇报 · 2026.09.26 · 数据截至报告快照

研究问题：特征放不进内存，训练如何持续推进?

目标是降低全量业务特征的常驻需求，同时保持训练输入语义。

问题约束

数据集业务特征无法一次载入 RAM
图拓扑仍常驻内存
训练节点特征按采样结果读取

正式比较范围

vanilla： 未引入后续优化的在线基线
v1： 单进程优化实现
v2： 当前最新多进程实现

batch 仅说明传统全量物化方式，不进入性能排名和加速比统计。

优化沿着关键路径逐层推进

先减少工作量，再增加并行度，最后压低复用与资源管理开销。

vanilla

建立按需 SQL 路径

v1 / 第一轮

查询修复 · 去重 · 缓存

v1 / 第二轮

encoded · direct · 线程

v2

多进程 · 共享 · 磁盘

v2 后续： HM 传输粒度 → Salt/Stack 共享查询 → mmap 与 WAL 长跑治理

历史文档中的 v1 是当时版本，不等于今天 relconnector/ 的全部代码。

GPU 等待特征，成为最明确的优化信号

历史瓶颈定位： ready queue 等待占训练时间的比例



特征生产： SQL → 解码 → 文本编码 → TensorFrame → GPU

阶段服务时间会重叠；判断瓶颈应结合 wall time 、 ready 等待和具体操作计时。

来源： doc/bottleneck-analysis-and-optimization.md | 历史版本，仅用于解释选题依据

第一轮优化：让 SQL 真正只取需要的行

保留零基 node ID 契约，在数据库查询边界使用可索引的 rowid 。

原始问题

WHERE / ORDER BY 使用 rowid − 1
表达式可能破坏直接 rowid 查找
少量采样请求触发大范围扫描

修复路径

WHERE rowid IN (...)
ORDER BY rowid
仅在 SELECT 输出中转换为零基 ID

进一步：稀疏请求合并 IN 查询，稠密访问按块读取并复用 raw cache 。

当前 vanilla 已回补 rowid 修复，旧版超时不能直接代表今天的 vanilla 。

去重的是特征生产，不是训练样本

批内、推荐分支及跨 batch window 合并静态特征请求，再恢复原顺序。

请求节点

[7, 3, 7, 8]

唯一节点

[3, 7, 8]

读取与编码

每个唯一节点处理一次

inverse 恢复

[f7, f3, f7, f8]

扩大复用范围

source / positive / negative 及相邻 batch

保持语义边界

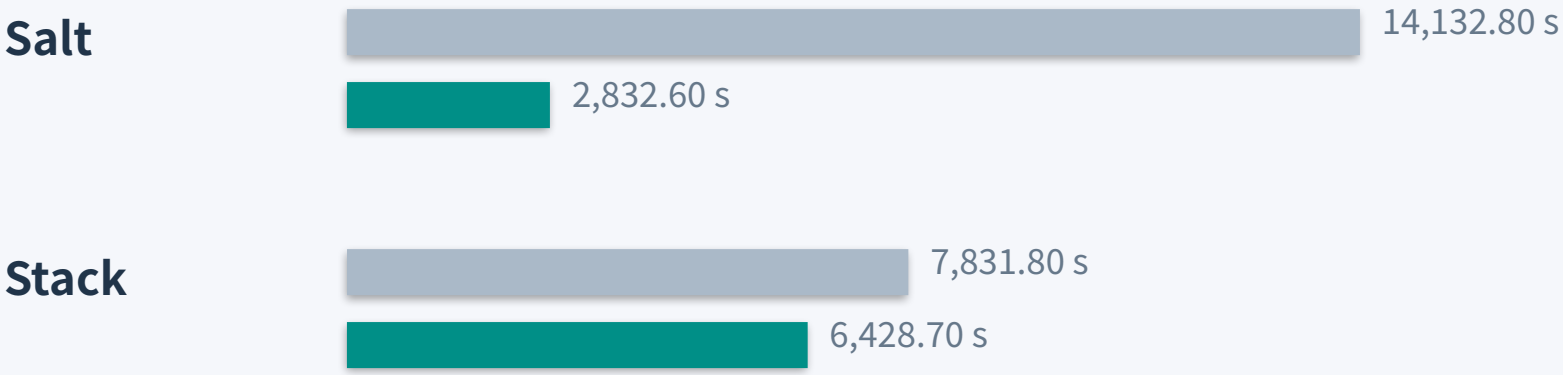
保留重复位置、 label 、边与 seed time

窗口受 batch / byte 上限约束；时间相关表示仍按各自 batch 计算。

第一轮收益取决于数据的重复结构

历史 vanilla 与早期 v1：同轮次 wall time，越短越好。

灰：历史 vanilla　绿：早期 v1



4.99× / 1.22×

Salt：高节点复用
Stack：低节点复用
整句 LRU 仍抖动

重复因子： Salt 2.114； Stack 1.011 。相同优化在不同工作集上的收益不同。

这些历史数字解释优化方向，不是当前三个版本的配对实验成绩。

正确性修订为后续性能比较建立共同契约

优化执行过程，保持官方特征类型、任务隐藏规则和采样语义。

输入一致

共同读取官方 stype artifact
按任务语义隐藏目标列
不以性能为由删列或改型

结构一致

保留节点 / 边顺序
保留重复项与缺失位置
只复用静态特征

数值验证

固定输入比较 TensorFrame
比较 loss 与参数更新
GPU 与调度变化单独分析

修订标识： relbench-correctness-v1 ； 修复前后的 loss / 性能记录不能混用。

第二轮 v1：把复用提升到最终静态特征

文本向量命中仍需组装； encoded cache 进一步省掉反复的 TensorFrame 构造。

RAW CACHE	解码后的 DataFrame 块	**省 SQL / 解码**
TEXT CACHE	冻结的 GloVe 向量	**省文本模型执行**
ENCODED CACHE	最终静态 CPU TensorFrame	**省 prepare / converter**

按 (table, node_id) 找到特征，仅编码 miss ，再按采样顺序恢复。

只缓存冻结 / 静态表示，不缓存会随训练参数更新的 GNN 隐状态。

降低执行开销，但不默认“并行越多越快”

direct GloVe 、连续 slab 、线程流水线、自适应窗口与初始化缓存。

机制

direct： 复用官方 tokenize / lookup / pool
slab： 减少小 Tensor 与 storage 对象
线程： SQL 与 encode 重叠
窗口：按重复率与缓存覆盖率调整

已有短程证据

文本微基准：局部约 1.59×
Salt GPU 2×50： 训练耗时 ↓ 34.0%
Ratebeer GPU 2×20： ↓ 8.5%
F1 CPU 小任务：反而退化约 1.5%

局部快路径和短程组合消融不能直接外推完整 epoch 的端到端收益。

v2：围绕特征生产重设进程边界

v1 单进程优化后， assemble 成为瓶颈； v2 将采样、组装、训练分进程执行。

采样进程
seed 流 + 共享拓扑

有界 plan
窗口与 inflight 限制

组装进程池
SQL + GloVe + Frame

训练主进程
ready → GPU 更新

worker 内： SQLite reader / raw cache / 字符串 / DataFrame
跨进程：采样计划与准备好的 tensor

减少 Python 数据对象跨进程传输，同时为每个 worker 的私有工作集计费。

来源： relconnector_v2/runtime/process_pipeline.py 与 process_api.py

调度要同时控制顺序、供给与资源

有界队列解决积压； strict / ready 控制训练顺序；许可调整活跃 worker 。

顺序

strict： 按序消费
慢窗口可能阻塞后续

ready： 轮内先就绪先训练
epoch gate 防止跨轮越界

供给

ready 饥饿 → 增加活跃数
高水位 → 减少活跃数

跨 epoch 可提前准备
单个窗口仍按 epoch 拆开

预算

各队列按字节设定限额
inflight 限制未完成窗口

预先 spawn 不等于按需分配
暂停 worker 不等于释放内存

正式确定性对照先用 strict； ready 的吞吐与训练轨迹影响另行报告。

v2 的复用范围：从 worker 内扩展到跨运行

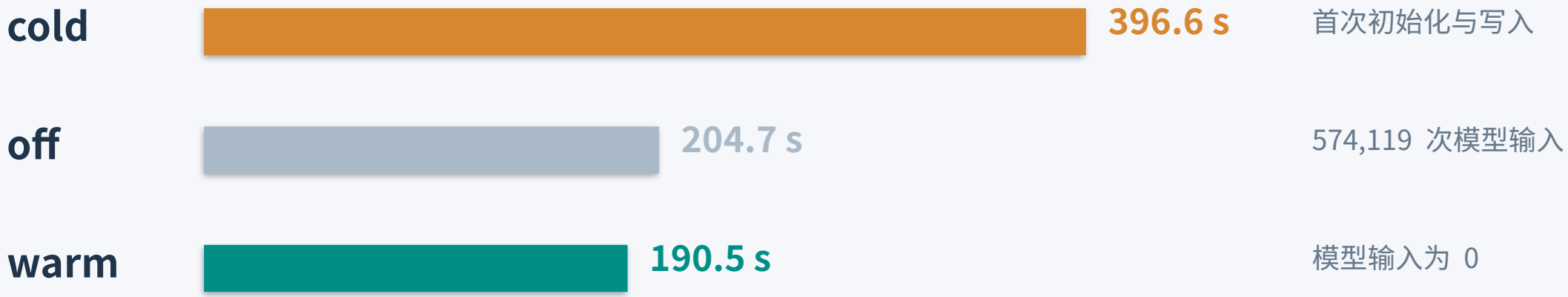
每层缓存解决不同的重复工作，也引入不同的管理成本。

worker 本地	raw / 可选 encoded / 共享槽位索引	快速访问，需控制每进程预算
运行期共享	文本向量 + reservation / encoded	避免跨 worker 重复编码
磁盘持久化	SQLite 索引 + float32 向量分片	复用超过 RAM 的文本工作集

磁盘缓存按访问惰性生成；不要求全表特征入 RAM，但新增长期磁盘成本。

磁盘缓存：模型调用归零，不等于总耗时归零

Arxiv 2×50 batch ｜磁盘缓存 cold / warm / off 的已有短测



warm 相比 off 仅快约 6.9%： frame conversion 、缓存查询与展开仍然存在。

cold / warm 的 2.08× 还含首次初始化成本，不能全部归因于避免文本编码。

HM： 高命中缓存，被整块 RPC 传输拖慢

减少传输粒度，再评估 worker 本地 encoded cache 。

根因 → 修复

article 命中率 99.77%
只请求少量行，却返回整块 TensorFrame

共享缓存仅传命中行
提供 local encoded ，避免反复 RPC

两轮 pipeline 时间

4,603 → 1,756 s

2.62×
ready 等待： 3,349 → 482 s

最终配置由 shared 2 GiB 改为 local 总计 4 GiB ；并非单一因素或等容量完整对照。

来源： doc/v2-hm-regression-20260925.md | 完整两轮 14,972 batch

Salt： 99.93% 命中，仍被共享查询拖慢

向量保持共享；本地只存已发布槽位索引，让重复访问绕过共享锁。

观察
查询成本 + 实际复用

→

高收益时启用
文本 → 共享槽位

→

重复命中
无共享锁批量 gather

→

低收益时退出
释放索引 + 冷却

18,260 → 5,208 s

3.51×

完整修复前后模型输入保持 146,635 / 0；没有靠减少 batch 获得提升。

3.51× 是修复旧 v2 的收益；相对历史 v1 的单位 batch 观测仅约 1.088× 。

来源： doc/v2-salt-regression-20260926.md | 完整两轮 6,340 batch

Stack： 从共享查询开销，到长跑资源耗尽

性能修复与可靠性修复，需要沿同一生产链路验证。

共享查询批量化

哈希移到锁外；元数据零拷贝视图
锁内取槽地址；锁外批量 gather
批量发布，限制取锁等待

100 batch： 296.04 → 53.45 s

长跑资源有界

每 cache 最多 1,024 个 mmap
LRU 关闭映射，不删除向量
同步错误通道避免报错时新建线程

完整 13,228 batch： 3,665.66 s

完整运行跨过旧挂起点并正常退出；早期无调用栈的停滞不能统一断言为死锁。

来源： doc/v2-stack-regression-20260926.md ｜暖磁盘缓存条件

Amazon： 资源回收也需要匹配磁盘工作负载

WAL 无节制增长与过于频繁 checkpoint ，都可能拖慢训练。

旧问题

WAL 约 60.46 GB

固定小阈值

64 MiB 频繁刷盘

自适应回收

按索引大小 / 空间调整

分片追加

每 writer 8,192 行 / 片

短测完成

2×100 batch： 676.01 s

资源回收

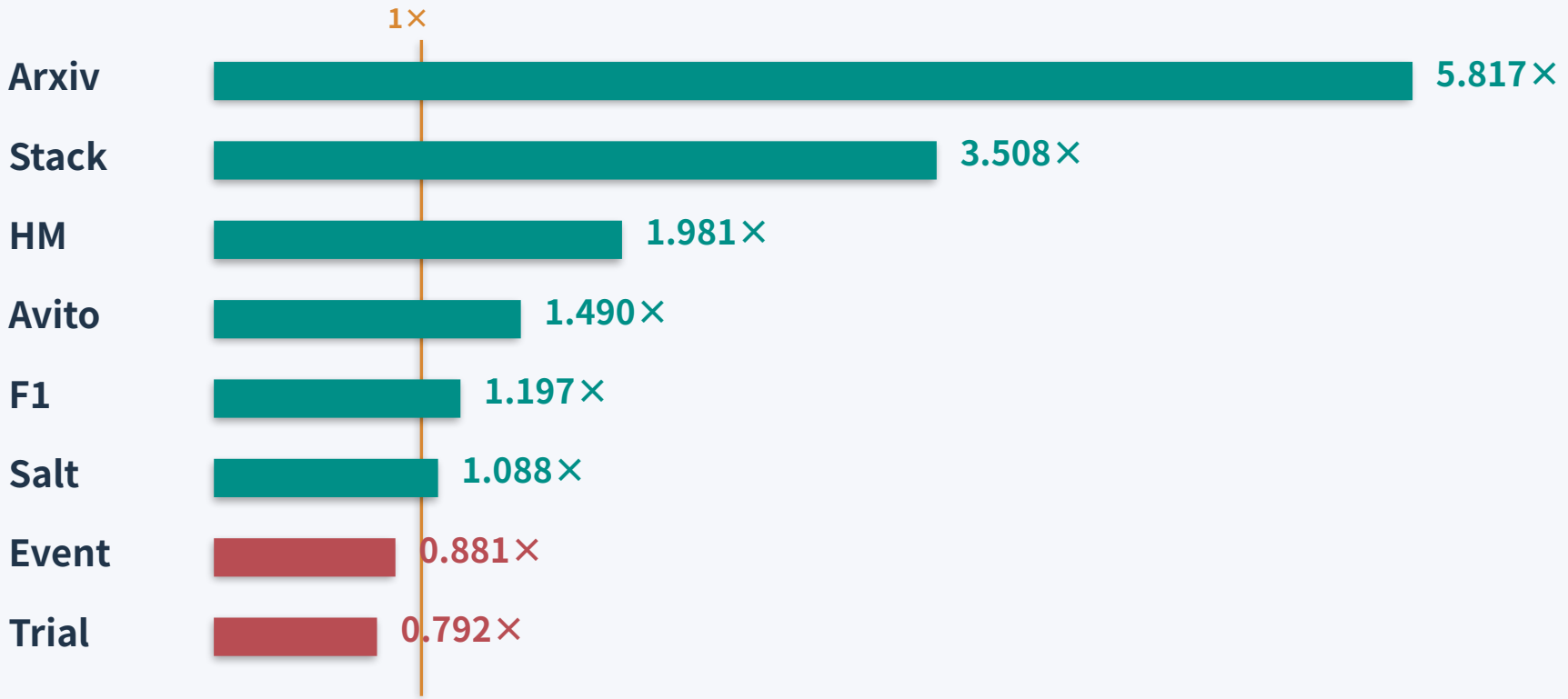
WAL 峰值约 1.35 GiB ，退出归零

这是资源回收短测；完整矩阵结果待补，续跑已使用部分热缓存。

来源： doc/v2-amazon-regression-20260926.md | 仅短程成功结果

当前结果：部分任务显著改善，仍有回归与缺口

历史 v1 / v2 的平均单位 batch 耗时比； >1 表示本组记录中 v2 更快。



比较限制

历史 A30 / v2 4090

1 epoch / 2 epochs

正确性与缓存差异

Amazon / Ratebeer

完整结果待补

这是历史趋势图，不是正式配对加速结论；不汇总“十任务总体加速”。

来源：报告 §8 / v2.jsonl | 8 个任务最新成功记录；不是统一最终源码重跑

有限内存验收：必须统计整个训练系统

v2 的 peak_rss_mb 当前使用 RUSAGE_SELF，只记录训练主进程。

主进程约 2–4 GiB ≠ 整个 v2 训练系统只需 2–4 GiB

额外内存

采样 / Assembly / manager
GloVe 与临时 Frame
共享索引、传输与在途数据

预算边界

ready 默认上限 16 GiB
队列容量并非实际峰值
暂停进程不释放全部内存

完整测量

进程树 PSS / USS
共享内存、 cgroup 峰值
显存、磁盘、 FD / mmap

100 万条 300 维 float32 向量仅 payload 约 1.12 GiB，磁盘索引与 WAL 另计。

下一轮实验：把机制证据变成严格系统结论

相同训练语义、相同硬件、相同总内存限制，主对照只保留三个在线版本。

01 固定边界

冻结 vanilla / v1 / v2
固定类型、隐藏列与数据版本
strict 先测，再测 ready
完整记录样本与 batch 数

02 区分冷热

全冷启动
初始化暖 / 文本冷
初始化暖 / 文本热
记录缓存初始与最终体积

03 做关键消融

1 / 2 / 4 worker：等总预算
encoded： off / shared / local
文本： shared / adaptive
磁盘： off / cold / warm

先补 Amazon / Ratebeer，诊断 Event / Trial，再统一最终代码重跑已完成任务。

核心贡献：让有限内存下的特征供给更高效

优化主线：减少重复工作 → 提高生产并行度 → 降低复用成本

按需 从全量特征驻留，转为按采样读取

复用 从原始行与文本，提升到最终静态特征

并行 用有界多进程让特征生产与 GPU 更新重叠

长跑 同时治理共享查询、 mmap 与 WAL 成本

现有机制与定向回归已有证据；统一版本的性能与总内存结论仍需配对验收。

附录 A ｜结果数值与工作量

报告快照中每任务最后一条成功记录； v2 两轮，历史 v1 一轮。

任务	v2 batches	v2 wall (s)	v2 s/batch	历史 v1 / v2
Arxiv	2,088	981.64	0.47013	5.817×
Avito	8,644	1,604.83	0.18566	1.490×
Event	76	2,193.96	28.86783	0.881×
F1	12	26.57	2.21388	1.197×
HM	14,972	1,760.13	0.11756	1.981×
Salt	6,340	5,208.23	0.82149	1.088×
Stack	13,228	3,665.66	0.27711	3.508×
Trial	918	656.95	0.71563	0.792×

Amazon 、 Ratebeer 未纳入。硬件、缓存、正确性版本与训练顺序不同，详见第 19 页。

来源： optimization-evolution-report-20260926.md §8； v2-full-matrix-20260925/v2.jsonl

附录 B ｜来源、复现与讲解备注

演示依据已完成的详细报告；每页备注记录来源与重要实验条件。

文档与实验依据

optimization-evolution-report-20260926.md
gpu-feature-optimization-experiment.md
adaptive-feature-optimization-report.md
v2-{hm,salt,stack,amazon}-regression-*
v2-disk-text-cache-experiment-20260925.md

源码与产物

relconnector_v2/runtime/process_pipeline.py
features/shared_text.py / shared_encoded.py
features/adaptive_text_index.py / disk_text.py
benchmarks/v2-full-matrix-20260925/v2.jsonl
本目录 build_slides.py 可重新生成 PPTX

本演示未重新运行训练；详细报告与历史记录中的测试成绩均保留原始口径。